

DAY 4

PostgreSQL 18 Backup & Disaster Recovery

Step-by-Step Lab Manual for a LIVE Patroni / Zalando Postgres Operator Cluster

Namespace	postgres-ha
Cluster	acid-postgres-ha (Zalando Operator + Patroni)
PostgreSQL	18
Backup tools	pg_dump, pg_dumpall, pg_basebackup, WAL archiving, pgBackRest
Enterprise backup	Full / Incremental / Differential / Compression / Encryption / Retention / PITR
Restore rule	Restore only to isolated recovery storage / separate recovery cluster

CRITICAL SAFETY RULE: Take backups from the live cluster, but never overwrite the active Patroni member PVCs during restore or PITR. Perform physical restore only on isolated recovery storage or a separate recovery cluster.

COPY/PASTE RULE: Every shell command in this corrected edition is a separate paragraph. Run commands one at a time. Do not paste two commands onto the same shell line unless they are intentionally joined with &&.

Table of Contents

- 0. Lab Architecture, Scope and Safety
- 1. Pre-Lab Validation of the Live Patroni Cluster
- 2. Backup Concepts: Logical, Physical, WAL and PITR
- 3. Lab 1 - Logical Backup from the Live Cluster
- 4. Lab 2 - Physical Backup with pg_basebackup (Corrected)
- 5. Lab 3 - WAL Archiving Validation
- 6. Lab 4 - Configure pgBackRest for the Live Cluster
- 7. Lab 5 - Full Backup with pgBackRest
- 8. Lab 6 - Incremental Backup
- 9. Lab 7 - Differential Backup
- 10. Repository, Compression, Encryption and Retention
- 11. Lab 8 - Restore / Full Cluster Recovery
- 12. Lab 9 - Point-in-Time Recovery (PITR)
- 13. Patroni Failover Considerations During Backup
- 14. Validation, Monitoring, Troubleshooting and Cleanup
- 15. RPO / RTO and Suggested Backup Schedule
- 16. Final Coverage Checklist

Implementation note: Zalando/Spilo commonly uses WAL-G/WAL-E for native backup. pgBackRest is not automatically present in all Spilo images. The pgBackRest portion therefore assumes an immutable image that contains a PostgreSQL 18-compatible pgBackRest build and a durable repository. Do not install packages ad hoc into a running production pod and assume they will survive restart.

0. Lab Architecture, Scope and Safety

0.1 What runs live and what is isolated?

- LIVE: acid-postgres-ha remains Patroni-managed and continues serving traffic during backup exercises.
- LIVE BACKUP: logical dumps, pg_basebackup, WAL archive checks and pgBackRest backup commands run against the current primary or an eligible replica.
- ISOLATED RESTORE: physical restore and PITR use separate recovery storage. Never replace files below active PGDATA while Patroni is running.
- REPOSITORY: use durable object storage (GCS/S3) or, for training only, an RWX volume reachable by every member.

0.2 Recovery architecture

```
Application -> acid-postgres-ha Service -> Patroni Primary
Primary -> logical backup / physical backup / WAL archive / pgBackRest repository
Repository + archived WAL -> isolated recovery PGDATA -> validation -> controlled cutover
```

0.3 Why failover is not backup

Patroni protects availability. It does not protect against logical mistakes such as DROP TABLE, accidental DELETE, bad application updates, corruption that propagates to replicas, or retention mistakes. Backup and PITR provide recovery from those events.

1. Pre-Lab Validation of the Live Patroni Cluster

1.1 Set namespace and inspect resources

```
kubectl config set-context --current --namespace=postgres-ha
kubectl get pods -o wide
kubectl get svc
kubectl get postgresql
kubectl get pvc
kubectl get statefulset
```

1.2 Set variables

```
NS=postgres-ha
CLUSTER=acid-postgres-ha
```

1.3 Identify the current primary - do not assume pod-0

```
for p in acid-postgres-ha-0 acid-postgres-ha-1 acid-postgres-ha-2; do echo "=== $p ==="; kubectl -n "$NS" exec "$p" -- curl -s http://127.0.0.1:8008/ | jq -r '"role=" + (.role // "unknown") + "state=" + (.state // "unknown")'; done
```

Exactly one member should report primary/leader. If the cluster is unhealthy, stop the backup lab and repair HA first.

1.4 Set PRIMARY and REPLICA after checking

```
PRIMARY=acid-postgres-ha-0
REPLICA=acid-postgres-ha-1
```

Change both values to match the actual current primary and a healthy replica.

1.5 Validate role, version, PGDATA and LSN

```
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -Atqc "SELECT pg_is_in_recovery(),  
current_database(), version();"
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -Atqc "SHOW data_directory;"
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -c "SELECT now(), pg_current_wal_lsn(),  
pg_is_in_recovery();"
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -c "SELECT application_name, client_addr,  
state, sync_state, sent_lsn, write_lsn, flush_lsn, replay_lsn FROM pg_stat_replication;"
```

2. Backup Concepts: Logical, Physical, WAL and PITR

Logical backup (pg_dump/pg_dumpall) stores SQL objects and data and supports database/object-level restore. Physical backup (pg_basebackup/pgBackRest) copies the cluster data files and is the foundation for cluster-level recovery and PITR. WAL archiving continuously preserves transaction history. PITR restores a physical backup and replays archived WAL up to a selected time/LSN/transaction.

2.1 Full vs differential vs incremental

- Full: complete physical backup; establishes an independent backup baseline.
- Differential: changes since the most recent full backup. Restore uses that full + the chosen differential + required WAL.
- Incremental: changes since the immediately preceding backup. Restore can require a chain of dependent backups + required WAL.
- Archived WAL is not a substitute for a base backup; it is the continuous roll-forward stream used after restoring a physical backup.

2.2 RPO and RTO

RPO is acceptable data loss. Frequent, healthy WAL archiving reduces potential data loss. RTO is acceptable service restoration time; it depends on backup size, storage/network speed, number of backup dependencies, WAL replay volume and validation time.

3. Lab 1 - Logical Backup from the Live Cluster

3.1 Create local backup directory

```
mkdir -p backups/logical
```

3.2 Full cluster logical backup

```
kubectl -n "$NS" exec "$PRIMARY" -- pg_dumpall -U postgres --clean --if-exists > "backups/logical/${CLUSTER}-dumpall-$(date +%Y%m%d-%H%M%S).sql"
ls -lh backups/logical/
```

3.3 Custom-format backup of banking database

```
kubectl -n "$NS" exec "$PRIMARY" -- pg_dump -U postgres -Fc -d banking -f /tmp/banking.dump
kubectl -n "$NS" cp "$PRIMARY:/tmp/banking.dump" "backups/logical/banking-$(date +%Y%m%d-%H%M%S).dump"
```

3.4 Verify without restoring

```
BACKUP_FILE=$(ls -lt backups/logical/banking-*.dump | head -1)
echo "$BACKUP_FILE"
pg_restore -l "$BACKUP_FILE" | head -40
SQL_FILE=$(ls -lt backups/logical/${CLUSTER}-dumpall-*.sql | head -1)
head -30 "$SQL_FILE"
```

3.5 Test logical restore into an isolated test database

```
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -c "DROP DATABASE IF EXISTS banking_restore_test;"
```

```
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -c "CREATE DATABASE banking_restore_test;"
BACKUP_FILE=$(ls -1t backups/logical/banking-*.dump | head -1)
echo "$BACKUP_FILE"
kubectl -n "$NS" cp "$BACKUP_FILE" "$PRIMARY:/tmp/banking-restore.dump"
kubectl -n "$NS" exec "$PRIMARY" -- pg_restore -U postgres -d banking_restore_test --clean --if-
exists /tmp/banking-restore.dump
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -d banking_restore_test -c "\dt"
```

On a real production system, run restore tests in a dedicated recovery cluster instead of consuming I/O/storage on the production primary.

4. Lab 2 - Physical Backup with pg_basebackup (Corrected)

This section corrects the original manual. pg_basebackup uses the PostgreSQL replication protocol. A normal superuser login is not enough if pg_hba.conf does not allow that user for replication. In this Zalando/Patroni cluster, use the cluster primary Service and the operator-managed replication/standby credentials.

4.1 Verify the primary Service name

```
kubectl -n "$NS" get svc
PRIMARY_SERVICE=acid-postgres-ha
kubectl -n "$NS" exec "$REPLICA" -- getent hosts "$PRIMARY_SERVICE"
```

Do not use -h 127.0.0.1 when the client command is running in a replica pod. 127.0.0.1 means that same replica pod. Do not use -h postgres-ha unless that Service actually exists. For this cluster, the primary Service is acid-postgres-ha.

4.2 Inspect replication state and lag

```
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -c "SELECT application_name, client_addr,
state, sync_state, sent_lsn, write_lsn, flush_lsn, replay_lsn,
pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), replay_lsn)) AS replay_lag FROM
pg_stat_replication;"
```

4.3 Identify replication-capable roles

```
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -c "SELECT rolname, rolreplication,
rolcanlogin FROM pg_roles WHERE rolreplication = true;"
```

4.4 Check replication HBA rules

```
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -c "SELECT line_number, type, database,
user_name, address, auth_method, error FROM pg_hba_file_rules WHERE 'replication' = ANY(database);"
```

Do not add broad rules such as host replication all 0.0.0.0/0 trust. Use the operator-managed replication role and existing HBA policy.

4.5 Find the operator-managed replication/standby secret

```
kubectl -n "$NS" get secrets | grep -Ei "standby|replic"
```

If the cluster exposes the standard Zalando standby credential secret, set it as follows. If the actual secret name differs, substitute the name returned by the previous command.

```
REPL_SECRET=standby.acid-postgres-ha.credentials.postgresql.acid.zalan.do
```

4.6 Load replication username and password into your local shell

```
REPL_USER=$(kubectl -n "$NS" get secret "$REPL_SECRET" -o jsonpath='{.data.username}' | base64 -d)
REPL_PASS=$(kubectl -n "$NS" get secret "$REPL_SECRET" -o jsonpath='{.data.password}' | base64 -d)
echo "$REPL_USER"
```

Do not echo the password during training or production. Keep it only in the shell variable and unset it after the exercise.

4.7 Test ordinary connectivity with the replication user

```
kubectl -n "$NS" exec "$REPLICA" -- env PGPASSWORD="$REPL_PASS" psql -h "$PRIMARY_SERVICE" -p 5432
-U "$REPL_USER" -d postgres -c "SELECT pg_is_in_recovery(), inet_server_addr(),
inet_server_port();"

```

Expected: `pg_is_in_recovery()` = false because acid-postgres-ha routes to the current primary.

4.8 Prepare backup directory in the client pod

```
kubectl -n "$NS" exec "$REPLICA" -- rm -rf /tmp/basebackup
kubectl -n "$NS" exec "$REPLICA" -- mkdir -p /tmp/basebackup

```

4.9 Run pg_basebackup with the correct host and replication user

```
kubectl -n "$NS" exec "$REPLICA" -- env PGPASSWORD="$REPL_PASS" pg_basebackup -h "$PRIMARY_SERVICE"
-p 5432 -U "$REPL_USER" -D /tmp/basebackup -Ft -z -Xs -P

```

Options: `-Ft` = tar format; `-z` = gzip; `-Xs` = stream WAL while the backup runs; `-P` = progress display.

4.10 Verify physical backup files

```
kubectl -n "$NS" exec "$REPLICA" -- ls -lh /tmp/basebackup
kubectl -n "$NS" exec "$REPLICA" -- find /tmp/basebackup -maxdepth 1 -type f -printf "%f %s
bytes\n"

```

Typical tar-format output includes `base.tar.gz` and WAL tar output when WAL streaming is included.

4.11 Copy the backup out to the workstation

```
mkdir -p backups/physical
PHYSICAL_DIR="backups/physical/basebackup-$(date +%Y%m%d-%H%M%S)"
kubectl -n "$NS" cp "$REPLICA:/tmp/basebackup" "$PHYSICAL_DIR"
find "$PHYSICAL_DIR" -maxdepth 2 -type f -ls | head -20

```

4.12 Remove password variable from the shell

```
unset REPL_PASS

```

4.13 Troubleshooting the exact errors seen in this lab

- could not translate host name "postgres-ha": the Service name is wrong or does not exist. Use `kubectl get svc` and set `PRIMARY_SERVICE=acid-postgres-ha`.
- no `pg_hba.conf` entry for replication connection ... user "postgres": the host is reachable, but postgres is not permitted for replication. Use the operator-managed replication/standby role and its Secret.
- no `pg_hba.conf` entry ... SSL encryption / no encryption: the server evaluated SSL and non-SSL HBA paths and neither matched that replication user/source. Do not bypass this with trust; fix role/HBA/authentication correctly.

5. Lab 3 - WAL Archiving Validation (continued)

These pages replace the original manual from this point onward. The first eight original pages, including Labs 0-2, remain at the front. Read the steps below in order; commands use your Ubuntu Bash terminal. If you open a new terminal, reset NS, CLUSTER, and PRIMARY as in original section 1.

5.1 Inspect archive settings

```
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
  psql -U postgres -c 'SHOW archive_mode;'
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
  psql -U postgres -c 'SHOW archive_command;'
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
  psql -U postgres -c 'SHOW archive_library;'
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
  psql -U postgres -x -c 'SELECT * FROM pg_stat_archiver;'
```

5.2 Force a WAL switch and check progress

```
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
  psql -U postgres -c 'SELECT pg_switch_wal();'
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
  psql -U postgres -x -c 'SELECT * FROM pg_stat_archiver;'
```

Archiving may take time; compare last_archived_wal and failed_count. Do not infer that a fixed sleep proves archival succeeded.

5.3 Check the existing Spilo archive mechanism

```
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
  sh -c 'command -v wal-g; command -v wal-e; command -v pgbackrest'
```

Spilo may already use WAL-G/WAL-E. Record what is running. Keep the current archive intact until a replacement repository, its pod credentials, and a full backup have been validated.

6. Lab 4 - Configure pgBackRest for the Live Cluster

This section preserves the original 6.1-6.9 sequence. The custom image exists, but live pgBackRest to GCS remains blocked: the non-GKE pods have no verified GCS identity, and your organization rejects service-account JSON key creation. Stop at the stated gate. The independent backup-lab-0 exercise in Labs 5-9 remains available.

6.1 Record current image, CR, StatefulSet and actual pod readiness

```
NS=postgres-ha
CLUSTER=acid-postgres-ha
IMAGE=docker.io/eswaribala/ericpostgrerepo2026:pg18
kubectl -n "$NS" get postgresql "$CLUSTER" \
  -o jsonpath='{.spec.dockerImage}{"\n"}'
kubectl -n "$NS" get sts "$CLUSTER" \
  -o jsonpath='{.spec.template.spec.containers[0].image}{"\n"}'
kubectl -n "$NS" get pods -o wide
kubectl -n "$NS" get postgresql "$CLUSTER" \
  -o yaml > "${CLUSTER}-before-day4.yaml"
```

The name is ericpostgrerepo2026, not ericpotgrerepo2026. All three database pods were already Ready with this image in your dry run. OnDelete does not support kubectl rollout status; check individual pod images and Ready flags. An "Edit cancelled" message means no CR change was saved.

6.2 Verify the existing immutable Docker image

```
for pod in acid-postgres-ha-0 acid-postgres-ha-1 acid-postgres-ha-2; do
  echo "$pod"
  kubectl -n "$NS" get pod "$pod" \
    -o jsonpath='{.spec.containers[0].image}{"\n"}'
  kubectl -n "$NS" exec "$pod" -c postgres -- \
    pgbackrest version
done
```

registry.example.com in the old Dockerfile was a placeholder, not a repository to use. The following image creation exercise restores every omitted step. Your current three pods already have the working pg18 image, so run 6.2.1-6.2.3 only when intentionally rebuilding. Use a NEW tag so the running pg18 tag is not silently overwritten.

6.2.1 Write the Dockerfile in your Ubuntu working directory

```
cat > Dockerfile.spilo-pgbackrest <<'EOF'
ARG BASE_IMAGE
FROM ${BASE_IMAGE}
USER root
RUN apt-get update \
  && apt-get install -y --no-install-recommends pgbackrest \
  && rm -rf /var/lib/apt/lists/*
USER postgres
EOF
cat Dockerfile.spilo-pgbackrest
```

For this repeat-build exercise, use the already verified pg18 image as the base. This is not a first-build recipe from an unknown upstream image. Before rebuilding, verify the base contains apt-get; stop if it does not.

6.2.2 Build and verify a NEW Docker Hub tag

```
docker login --username eswaribala
BASE_IMAGE=docker.io/eswaribala/ericpostgrrepo2026:pg18
NEW_IMAGE=docker.io/eswaribala/ericpostgrrepo2026:pg18-rebuilt
docker pull "$BASE_IMAGE"
docker run --rm --entrypoint sh "$BASE_IMAGE" \
  -c 'command -v apt-get && command -v pgbackrest'
docker build --build-arg BASE_IMAGE="$BASE_IMAGE" \
  -f Dockerfile.spilo-pgbackrest \
  -t "$NEW_IMAGE" .
docker run --rm --entrypoint pgbackrest \
  "$NEW_IMAGE" version
docker image inspect "$NEW_IMAGE" \
  --format '{{.Id}}'
```

If docker build fails at apt-get, inspect the base-image OS before changing package commands. This new tag is for an intentional rebuild; it does not automatically change the live PostgreSQL pods.

6.2.3 Push the built tag and inspect its digest

```
docker push "$NEW_IMAGE"
docker image inspect "$NEW_IMAGE" \
  --format '{{json .RepoDigests}}'
```

Docker Hub holds the container image. GCS is a separate service for backup data. A successful docker push does not grant your pods access to GCS.

6.2.4 Set up pull credentials if the Docker Hub repository is private

```
kubectl -n "$NS" get secret dockerhub-secret
kubectl -n "$NS" get sts "$CLUSTER" \
  -o jsonpath='{.spec.template.spec.imagePullSecrets}{"\n"}'
kubectl -n "$NS" get pod "$CLUSTER-0" \
  -o jsonpath='{.spec.serviceAccountName}{"\n"}'
```

The screenshot does not establish whether Docker Hub is private. If the secret exists and the pods already pull the custom image, keep it. If the repository is private and there is no working pull Secret, create one with a Docker Hub access token read without echoing it:

```
read -rsp 'Docker Hub access token: ' DOCKERHUB_TOKEN
echo
kubectl -n "$NS" create secret docker-registry dockerhub-secret \
  --docker-server=https://index.docker.io/v1/ \
  --docker-username=eswaribala \
  --docker-password="$DOCKERHUB_TOKEN" \
  --dry-run=client -o yaml | kubectl -n "$NS" apply -f -
unset DOCKERHUB_TOKEN
```

Attach dockerhub-secret through the operator-supported pod-template or service-account setting without replacing any existing imagePullSecrets. A private image pull Secret and a GCS workload identity solve different problems.

6.2.5 Change spec.dockerImage ONLY if deploying the rebuilt tag

```
kubectl -n "$NS" edit postgresql "$CLUSTER"
kubectl -n "$NS" get postgresql "$CLUSTER" \
  -o jsonpath='{.spec.dockerImage}{"\n"}'
kubectl -n "$NS" get sts "$CLUSTER" \
  -o jsonpath='{.spec.template.spec.containers[0].image}{"\n"}'
```

Inside the editor, locate the existing spec: block and set its dockerImage value to the pushed NEW_IMAGE exactly. Do not change metadata.name, teamId, volume, or the database stanza. If the editor reports "Edit cancelled, no changes made", nothing changed; inspect the CR again.

```
spec:
  dockerImage: docker.io/eswaribala/ericpostgrerepo2026:pg18-rebuilt
```

This is only a YAML fragment to edit into the existing PostgreSQL resource; do not paste it at a Bash prompt. Skip this entire edit when retaining the currently healthy pg18 image.

6.2.6 Roll replicas carefully under OnDelete and verify each image

The StatefulSet update strategy is OnDelete. The new template does not update existing pods until they are replaced. Confirm the current primary first, replace one healthy replica at a time, and wait for it to become Ready before considering another. Arrange a controlled primary switchover through the operator; never delete all three pods or use kubectl rollout status on an OnDelete StatefulSet.

```
kubectl -n "$NS" get sts "$CLUSTER" \
  -o jsonpath='{.spec.template.spec.containers[0].image}{"\n"}'
kubectl -n "$NS" get pods -o wide
```

Only if the StatefulSet template shows your newly pushed image and you have chosen to roll it out, find one healthy replica dynamically:

```
REPLICA=
for pod in acid-postgres-ha-0 acid-postgres-ha-1 acid-postgres-ha-2; do
  role=$(kubectl -n "$NS" exec "$pod" -c postgres -- \
    psql -U postgres -Atqc 'SELECT pg_is_in_recovery()')
  echo "$pod in_recovery=$role"
  if test "$role" = t && test -z "$REPLICA"; then
    REPLICA="$pod"
  fi
done
test -n "$REPLICA" && echo "Selected replica: $REPLICA"
```

Stop if no replica is selected, its database is not Ready, or it is the primary. When the new image is confirmed in the template and the selected replica is healthy, replace only that pod:

```
kubectl -n "$NS" get pod "$REPLICA"
```

```
kubectl -n "$NS" delete pod "$REPLICA"
kubectl -n "$NS" wait --for=condition=Ready \
  "pod/$REPLICA" --timeout=10m
kubectl -n "$NS" get pod "$REPLICA" \
  -o jsonpath='{.spec.containers[0].image}{"\n"}'
kubectl -n "$NS" exec "$REPLICA" -c postgres -- \
  pgbackrest version
```

If this succeeds, select the other replica and repeat one at a time. The current primary requires an operator-supported controlled switchover, which must be verified for your operator version before performing it. Do not delete it simply to update its image.

```
kubectl -n "$NS" get sts "$CLUSTER" \
  -o jsonpath='{.spec.updateStrategy.type}{"\n"}'
for pod in acid-postgres-ha-0 acid-postgres-ha-1 acid-postgres-ha-2; do
  echo "$pod"
  kubectl -n "$NS" get pod "$pod" \
    -o jsonpath='{.spec.containers[0].image}{" "}{.status.phase}{"\n"}'
  kubectl -n "$NS" exec "$pod" -c postgres -- \
    pgbackrest version
done
```

If the image in all three pods is already `docker.io/eswaribala/ericpostgrerepo2026:pg18` and `pgbackrest` version works, the image lab is complete: skip rebuild, push, CR editing, and pod replacement. The six missing-file checks in your screenshot concern mounts, not the image.

6.3 Create a repository encryption Secret only after repository selection

The original Secret protects the planned live GCS repository; it does not encrypt backup-lab-0. Preserve a secure external copy of the passphrase. Do not generate a fresh passphrase over an existing encrypted repository.

```
kubectl -n "$NS" get secret pgbackrest-repo-secret
```

If the Secret does not exist, create one when you have approved repository access. Do not run create again if it already exists; an encrypted repository must keep its original passphrase. `CONDITIONAL` command after identity and repository approval:

```
PGBR_CIPHER_PASS=$(openssl rand -base64 48)
kubectl -n "$NS" create secret generic pgbackrest-repo-secret \
  --from-literal=repo-cipher-pass="$PGBR_CIPHER_PASS"
unset PGBR_CIPHER_PASS
```

6.4 Discover actual live PGDATA

```
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
  psql -U postgres -Atqc 'SHOW data_directory;'
```

Copy this output into `pg1-path` when a live stanza is configured. Do not guess `/home/postgres/pgdata/pgroot/data`.

6. Lab 4 - Live GCS preparation and identity gate

6.5 Verify selected project and existing bucket

```
gcloud auth list
gcloud config get-value project
PROJECT_ID=$(gcloud config get-value project)
GCS_BUCKET=postgres-ha-backups-parameswari-2026
echo "$PROJECT_ID"
echo "$GCS_BUCKET"
gcloud storage buckets describe "gs://$GCS_BUCKET"
```

Use the project ID, not its display name. If the bucket does not exist, creation requires permissions and active billing. Creating a bucket does not authorize pods to access it.

```
gcloud billing projects describe "$PROJECT_ID"
gcloud storage buckets create "gs://$GCS_BUCKET" \
  --project="$PROJECT_ID" --location=asia-south1 \
  --default-storage-class=STANDARD \
  --uniform-bucket-level-access \
  --public-access-prevention
```

Run the create command only if describe said the name is available and the project has billing enabled. Bucket names are globally unique; change GCS_BUCKET if another project owns that name.

6.5.1 Gate - authenticate non-GKE Kubernetes pods

Your organization enforces constraints/iam.disableServiceAccountKeyCreation. Do not run gcloud iam service-accounts keys create. gcloud auth login authenticates your Ubuntu shell, not Kubernetes pods. GCS config `repo1-gcs-key-type=auto` requires a working pod identity; do not assume that setting creates one. An administrator must configure an approved identity federation mechanism or select another durable, failover-safe repository. Test write and read from the same image and service account used by the database before continuing.

6.5.2 ConfigMap and encryption Secret after the identity gate

Keep the original goals: create one pgBackRest stanza with discovered PGDATA, GCS bucket/path, retention, compression, and encryption. The following is CONDITIONAL after successful pod-level bucket authentication. The same chosen stanza name must be used throughout live backup and recovery. A ConfigMap in the namespace does not prove any pod mounts it.

```
PGDATA_ACTUAL=$(kubectl -n "$NS" exec "$PRIMARY" \
  -c postgres -- psql -U postgres -Atqc \
  'SHOW data_directory;')
test -n "$PGDATA_ACTUAL" && test -n "$GCS_BUCKET"
cat > pgbackrest.conf <<EOF
[postgres-ha]
pg1-path=$PGDATA_ACTUAL
pg1-port=5432

[global]
repo1-type=gcs
repo1-gcs-bucket=$GCS_BUCKET
repo1-gcs-key-type=auto
repo1-path=/pgbackrest/postgres-ha
repo1-retention-full=2
repo1-cipher-type=aes-256-cbc
compress-type=zst
compress-level=3
log-level-console=info
EOF
kubectl -n "$NS" create configmap pgbackrest-config \
  --from-file=pgbackrest.conf=./pgbackrest.conf
```

This uses `repo1-gcs-key-type=auto` ONLY if an approved Kubernetes workload identity already works from the target pod. Creating this ConfigMap without that identity does not fix GCS 403 failures. Do not create a second cipher Secret with a changed value.

6.6 Mount through the PostgreSQL operator CR

Use your installed Zalando operator CRD `additionalVolumes` fields for `pgbackrest.conf`, `repo` Secret, and any required pod identity. Never edit the generated StatefulSet as the source of truth. Do not apply changes while the GCS gate is failing. The `spec.dockerImage` already points at the verified image; `metadata.name` and the future stanza name do not change. CONDITIONAL editor workflow:

```
kubectl explain postgresql.spec.additionalVolumes
kubectl -n "$NS" edit postgresql "$CLUSTER"
kubectl -n "$NS" get postgresql "$CLUSTER" \
  -o jsonpath='{.spec.additionalVolumes}{"\n"}'
kubectl -n "$NS" get sts "$CLUSTER" \
  -o jsonpath='{.spec.template.spec.volumes}{"\n"}'
```

For CRDs that support `additionalVolumes`, place the following under existing `spec`: in the editor. This is YAML content, not a terminal command. Preserve any other `additionalVolumes` and all existing `spec` keys:

```
additionalVolumes:
- name: pgbackrest-config
  mountPath: /etc/pgbackrest
  volumeSource:
    configMap:
      name: pgbackrest-config
- name: pgbackrest-repo-secret
  mountPath: /etc/pgbackrest-secret
  volumeSource:
    secret:
      secretName: pgbackrest-repo-secret
```

Before touching pods, check that the operator-generated StatefulSet template has the mounts. For this OnDelete StatefulSet, plan controlled replacement of one replica at a time, verifying readiness and Patroni role. Do not delete the primary or several members to force an update.

6.7 Check all three Patroni pods

```
for pod in acid-postgres-ha-0 acid-postgres-ha-1 acid-postgres-ha-2; do
  kubectl -n "$NS" get pod "$pod" \
    -o jsonpath='{.metadata.name}{ " "}{.status.phase}{"\n"}'
  kubectl -n "$NS" exec "$pod" -c postgres -- \
    pgbackrest version
done
```

Your screenshot shows exit code 1 for all six file checks: `pgbackrest.conf` and `repo-cipher-pass` are not readable in the three live pods. Do not repeat the file test now. First inspect whether the ConfigMap/Secret exist and whether the operator CR and StatefulSet template contain their mounts:

```
kubectl -n "$NS" get configmap pgbackrest-config
kubectl -n "$NS" get secret pgbackrest-repo-secret
kubectl -n "$NS" get postgresql "$CLUSTER" \
  -o jsonpath='{.spec.additionalVolumes}{"\n"}'
kubectl -n "$NS" get sts "$CLUSTER" \
  -o jsonpath='{.spec.template.spec.volumes}{"\n"}'
```

These are read-only checks. Stop after them: do not mount an unverified GCS config into the live pods or replace pods to make the test pass. When a future approved repository and pod identity have passed the gate, verify both files in EVERY newly updated pod as a conditional final check:

```
for pod in acid-postgres-ha-0 acid-postgres-ha-1 acid-postgres-ha-2; do
  kubectl -n "$NS" exec "$pod" -c postgres -- \
    test -r /etc/pgbackrest/pgbackrest.conf
```

```
kubectl -n "$NS" exec "$pod" -c postgres -- \
  test -r /etc/pgbackrest-secret/repo-cipher-pass
done
```

6.8 Stanza creation and repository check

Run stanza-create/check ONLY after proven pod-level repository access, mounted config, original passphrase, and all-member validation. Do not run --stanza=lab on acid-postgres-ha; lab belongs to the separate backup-lab-0 database. CONDITIONAL commands:

```
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- sh -ec \
'export PGBACKREST_REP01_CIPHER_PASS=$(cat /etc/pgbackrest-secret/repo-cipher-pass)";
pgbackrest --stanza=postgres-ha stanza-create;
pgbackrest --stanza=postgres-ha info'
```

6.9 WAL archive change

Do not change Patroni archive_command yet. Keep the existing working WAL archive. Once the new stanza is proven, plan a controlled archive migration with the operator CR or patronictl dynamic config (depending on your installed operator), switch WAL, inspect pg_stat_archiver and pgbackrest check, then take and restore a full backup. Never edit postgresql.conf directly in an operator-managed pod. Use a controlled restart if archive_mode changes. A sample archive_command is:

```
archive_command: >-
PGBACKREST_REP01_CIPHER_PASS=$(cat /etc/pgbackrest-secret/repo-cipher-pass)
pgbackrest --stanza=postgres-ha archive-push %p
```

The command is a PostgreSQL configuration VALUE, not something to paste into your terminal. Its feasibility depends on mounts, user permissions and the existing archive mechanism; verify with SHOW archive_command, pg_stat_archiver, a WAL switch, and pgbackrest check before relying on it.

7. Lab 5 - Full Backup with pgBackRest

The original live --stanza=postgres-ha commands depend on completing all of Lab 4. For your current dry run, take a full backup of the separate backup-lab-0 database using its existing /repo PVC and stanza lab. This is a training repository on local-path storage, not off-node disaster recovery.

```
kubectl -n "$NS" exec backup-lab-0 -- \
  gosu postgres pgbackrest --stanza=lab info
kubectl -n "$NS" exec backup-lab-0 -- \
  gosu postgres pgbackrest --stanza=lab --type=full backup
kubectl -n "$NS" exec backup-lab-0 -- \
  gosu postgres pgbackrest --stanza=lab info
```

You already had a successful full backup in this repository. A new full is optional for a fresh exercise. A new full label ends F. Do not insert a second -- after postgres: gosu postgres pgbackrest is the executable sequence.

8. Lab 6 - Incremental Backup

```
kubectl -n "$NS" exec backup-lab-0 -- \
  gosu postgres pgbackrest --stanza=lab --type=incr backup
kubectl -n "$NS" exec backup-lab-0 -- \
  gosu postgres pgbackrest --stanza=lab info
```

The 28 September incremental already completed successfully with label 20260926-161449F_20260928-174132I. A new incremental is optional. Its label ends I and depends on earlier backups.

9. Lab 7 - Differential Backup

```
kubectl -n "$NS" exec backup-lab-0 -- \
  gosu postgres pgbackrest --stanza=lab --type=diff backup
kubectl -n "$NS" exec backup-lab-0 -- \
  gosu postgres pgbackrest --stanza=lab info
kubectl -n "$NS" exec backup-lab-0 -- \
  gosu postgres pgbackrest --stanza=lab check
```

A differential label ends D and includes changes since the latest full backup. The existing repo already has a prior differential. Only the separate lab database is protected by these sets.

10. Repository, Compression, Encryption and Retention

10.1 Repository

The original proposed GCS repository for the live Patroni cluster has no working pod identity today. The working /repo PVC belongs only to the standalone backup-lab-0 pod. Its rancher.io/local-path storage is on one node and is not a shared off-node recovery design.

10.2 Compression

Inspect backup-lab-config for its actual compression setting; do not assume the proposed zst setting applies to this repository.

```
kubectl -n "$NS" get configmap backup-lab-config -o yaml
```

10.3 Encryption

Your pgbackrest info screenshot showed cipher: none for stanza lab. The original aes-256-cbc and cipher Secret settings describe a future live repository; they are not enabled by creating a Kubernetes Secret alone. Keep the original passphrase outside the cluster if you configure encryption later.

10.4 Retention

The standalone config uses its own repo1-retention-full policy. Full, differential and incremental backup chains depend on one another. Changing retention can expire older sets.

10.5 Inspect before expiration

```
kubectl -n "$NS" exec backup-lab-0 -- \
gosu postgres pgbackrest --stanza=lab info
```

Do not run expire just to complete the lab. Preserve the chain needed for the restore and PITR exercises.

11. Lab 8 - Restore / Full Cluster Recovery

Preserve the original 11.1-11.6 objectives: create a distinct recovery target, discover its PVC/image, prepare a toolbox, validate empty PGDATA, restore the latest set, and start PostgreSQL for SQL verification. The original procedure first bootstrapped an operator-managed recovery cluster and then overwrote its PVC. Use a NEW PVC and isolated toolbox instead. None of the active acid-postgres-ha PVCs is a restore target.

11.1 Check source configuration and create a NEW recovery PVC

```
kubectl -n "$NS" get configmap backup-lab-config \
-o jsonpath='{.data.pgbackrest\conf}'
kubectl -n "$NS" get pvc backup-lab-repo
cat > day4-lab-restore-pvc.yaml <<'YAML'
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: day4-lab-restore-data
  namespace: postgres-ha
spec:
  accessModes: [ReadWriteOnce]
  resources:
    requests:
      storage: 3Gi
YAML
kubectl apply -f day4-lab-restore-pvc.yaml
```

Gate: config must include [lab], repo1-path=/repo and the PostgreSQL 18 data path. The target PVC size must exceed the actual source data size. WaitForFirstConsumer may leave it Pending until the toolbox is scheduled.

11.2 Discover source image, node and postgres UID/GID

```
LAB_IMAGE=$(kubectl -n "$NS" get pod backup-lab-0 \
-o jsonpath='{.spec.containers[0].image}')
LAB_NODE=$(kubectl -n "$NS" get pod backup-lab-0 \
-o jsonpath='{.spec.nodeName}')
LAB_HOST=$(kubectl get node "$LAB_NODE" \
-o jsonpath='{.metadata.labels.kubernetes.io/hostname}')
LAB_UID=$(kubectl -n "$NS" exec backup-lab-0 -- id -u postgres)
LAB_GID=$(kubectl -n "$NS" exec backup-lab-0 -- id -g postgres)
test -n "$LAB_IMAGE" && test -n "$LAB_HOST"
test -n "$LAB_UID" && test -n "$LAB_GID"
```

11.3 Create toolbox with a separate target and read-only repo

```
cat > day4-lab-restore-pod.yaml <<YAML
apiVersion: v1
kind: Pod
metadata:
  name: day4-lab-restore
  namespace: postgres-ha
spec:
  restartPolicy: Never
  nodeSelector:
    kubernetes.io/hostname: "$LAB_HOST"
  initContainers:
    - name: prepare-directory
      image: "$LAB_IMAGE"
      command: ["sh", "-ec"]
      args: ["mkdir -p /recovery/data; chown -R $LAB_UID:$LAB_GID /recovery/data"]
      securityContext: {runAsUser: 0}
      volumeMounts:
        - {name: target, mountPath: /recovery}
  containers:
    - name: toolbox
      image: "$LAB_IMAGE"
      command: ["sh", "-c", "sleep infinity"]
      volumeMounts:
        - {name: target, mountPath: /recovery}
        - {name: repo, mountPath: /repo, readOnly: true}
        - name: config
          mountPath: /etc/pgbackrest/pgbackrest.conf
          subPath: pgbackrest.conf
          readOnly: true
  volumes:
    - name: target
      persistentVolumeClaim: {claimName: day4-lab-restore-data}
    - name: repo
      persistentVolumeClaim: {claimName: backup-lab-repo}
    - name: config
      configMap: {name: backup-lab-config}
YAML
kubectl apply -f day4-lab-restore-pod.yaml
kubectl -n "$NS" wait --for=condition=Ready \
pod/day4-lab-restore --timeout=180s
```

If pod readiness fails, inspect pod events and both PVCs. Do not switch to a live data PVC. The repo is ReadWriteOnce; this exercise assumes the same-node local-path training cluster.

11. Lab 8 - Restore / Full Cluster Recovery (continued)

11.4 Check the recovery directory is empty and the repo is readable

```
kubectl -n "$NS" exec day4-lab-restore -c toolbox -- \
  sh -c 'test ! -e /recovery/data/PG_VERSION'
kubectl -n "$NS" exec day4-lab-restore -c toolbox -- \
  gosu postgres pgbackrest --stanza=lab info
```

11.5 Restore the latest valid set (omit invalid --set=latest)

```
kubectl -n "$NS" exec day4-lab-restore -c toolbox -- \
  gosu postgres pgbackrest --stanza=lab \
  --pg1-path=/recovery/data restore
kubectl -n "$NS" exec day4-lab-restore -c toolbox -- \
  cat /recovery/data/PG_VERSION
```

Expected: command completes successfully and PG_VERSION displays 18. No recovery operation touches the running Patroni cluster. PG_VERSION alone does not prove startup.

11.6 Start and query isolated restored PostgreSQL

```
kubectl -n "$NS" exec day4-lab-restore -c toolbox -- \
  gosu postgres pg_ctl -D /recovery/data \
  -o "-c listen_addresses='' -c port=5544 -c unix_socket_directories=/tmp" \
  -l /recovery/data/day4-recovery.log -w start
kubectl -n "$NS" exec day4-lab-restore -c toolbox -- \
  gosu postgres psql -h /tmp -p 5544 -U postgres \
  -d postgres -c 'SELECT current_database(), pg_is_in_recovery();'
kubectl -n "$NS" exec day4-lab-restore -c toolbox -- \
  gosu postgres pg_ctl -D /recovery/data -m fast -w stop
```

If startup fails, inspect /recovery/data/day4-recovery.log in the toolbox. No operator StatefulSet is attached to the restored PVC. Keep this PVC for inspection; deleting a local-path PVC may remove its data.

12. Lab 9 - Point-in-Time Recovery (PITR)

PITR needs a fresh full backup, archived WAL through the target, and another NEW target PVC. All SQL below runs in the standalone backup-lab-0 database, not the live banking database.

12.1 Full backup before the test transaction

```
kubectl -n "$NS" exec backup-lab-0 -- \
  gosu postgres pgbackrest --stanza=lab --type=full backup
```

12.2 Write good row and capture UTC target

```
kubectl -n "$NS" exec backup-lab-0 -- \
  gosu postgres psql -U postgres -d postgres \
  -c 'CREATE TABLE IF NOT EXISTS dr_day4_demo
      (id integer PRIMARY KEY, note text);'
kubectl -n "$NS" exec backup-lab-0 -- \
  gosu postgres psql -U postgres -d postgres \
  -c "INSERT INTO dr_day4_demo VALUES (280926, 'keep this')
      ON CONFLICT (id) DO UPDATE SET note=EXCLUDED.note;"
TARGET_TIME=$(kubectl -n "$NS" exec backup-lab-0 -- \
  gosu postgres psql -U postgres -d postgres -Atqc \
  "SELECT to_char(clock_timestamp() AT TIME ZONE 'UTC',
    'YYYY-MM-DD HH24:MI:SS.US') || '+00';")
echo "$TARGET_TIME"
test -n "$TARGET_TIME"
```

12.3 Delete test row and force WAL archive

```
kubectl -n "$NS" exec backup-lab-0 -- \
  gosu postgres psql -U postgres -d postgres \
  -c 'DELETE FROM dr_day4_demo WHERE id=280926;'
kubectl -n "$NS" exec backup-lab-0 -- \
  gosu postgres psql -U postgres -d postgres \
  -c 'SELECT pg_switch_wal();'
kubectl -n "$NS" exec backup-lab-0 -- \
  gosu postgres pgbackrest --stanza=lab check
```

12.4 Create the NEW PITR target (never reuse the restored PVC)

```
sed 's/day4-lab-restore-data/day4-lab-pitr-data/g' \
  day4-lab-restore-pvc.yaml > day4-lab-pitr-pvc.yaml
sed -e 's/day4-lab-restore-data/day4-lab-pitr-data/g' \
  -e 's/day4-lab-restore/day4-lab-pitr/g' \
  day4-lab-restore-pod.yaml > day4-lab-pitr-pod.yaml
kubectl apply -f day4-lab-pitr-pvc.yaml
kubectl apply -f day4-lab-pitr-pod.yaml
kubectl -n "$NS" wait --for=condition=Ready \
  pod/day4-lab-pitr --timeout=180s
kubectl -n "$NS" exec day4-lab-pitr -c toolbox -- \
  sh -c 'test ! -e /recovery/data/PG_VERSION'
```

Check the generated manifests before applying them. They must name day4-lab-pitr-data, never data-backup-lab-0 or any pgdata-acid-postgres-ha claim.

12.5 Restore to recorded target time

```
test -n "$TARGET_TIME"
kubectl -n "$NS" exec day4-lab-pitr -c toolbox -- \
  gosu postgres pgbackrest --stanza=lab \
  --pg1-path=/recovery/data --type=time \
  --target="$TARGET_TIME" --target-action=promote restore
```

12.6 Start the isolated instance and verify the good row survived

```
kubectl -n "$NS" exec day4-lab-pitr -c toolbox -- \
  gosu postgres pg_ctl -D /recovery/data \
  -o "-c listen_addresses='' -c port=5545 -c unix_socket_directories=/tmp" \
  -l /recovery/data/day4-pitr.log -w start
kubectl -n "$NS" exec day4-lab-pitr -c toolbox -- \
  gosu postgres psql -h /tmp -p 5545 -U postgres \
  -d postgres \
  -c 'SELECT id,note FROM dr_day4_demo WHERE id=280926;'
kubectl -n "$NS" exec day4-lab-pitr -c toolbox -- \
  gosu postgres pg_ctl -D /recovery/data -m fast -w stop
```

Expected row: keep this. If WAL is missing, pgBackRest restore or PostgreSQL startup will fail: inspect the log rather than claiming a successful PITR.

13. Patroni Failover Considerations During Backup

Recompute PRIMARY after switchover. Never assume pod 0 is still the leader. If a backup begins on a former primary, inspect pgBackRest output and restart the exercise only after verifying role and repository consistency. The lab backup-lab-0 pod is independent of Patroni.

```
PRIMARY=
for pod in acid-postgres-ha-0 acid-postgres-ha-1 acid-postgres-ha-2; do
  value=$(kubectl -n "$NS" exec "$pod" -c postgres -- \
    psql -U postgres -Atqc 'SELECT pg_is_in_recovery()')
  test "$value" = f && PRIMARY="$pod"
done
test -n "$PRIMARY" && echo "$PRIMARY"
```

14. Validation, Monitoring, Troubleshooting and Cleanup

14.1 Daily validation

```
kubectl -n "$NS" get pods -o wide
kubectl -n "$NS" exec backup-lab-0 -- \
  gosu postgres pgbackrest --stanza=lab check
kubectl -n "$NS" exec backup-lab-0 -- \
  gosu postgres pgbackrest --stanza=lab info
kubectl -n "$NS" exec "$PRIMARY" -c postgres -- \
  psql -U postgres -x -c 'SELECT * FROM pg_stat_archiver;'
```

These are two distinct archive/backup checks. The live Patroni GCS stanza is not configured simply because the separate lab stanza says ok. Error 031 means run pgBackRest as postgres. An extra -- after gosu postgres is interpreted as an executable and fails.

14.2 Cleanup only your disposable recovery objects

Keep the recovery PVCs while reviewing the result. Before deleting any PVC, confirm its exact name and the local-path reclaim behavior. Never delete pgdata-acid-postgres-ha-0/1/2, backup-lab-repo, or data-backup-lab-0 as generic cleanup. Do not create or delete an operator-managed recovery cluster for these isolated restores.

15. RPO / RTO and Suggested Backup Schedule

The original schedule is illustrative: continuous WAL archive; weekly full, daily differential, several incremental backups per day, optional logical dumps, and monthly restore drills. Actual RPO is limited by how much WAL was successfully archived; actual RTO includes restore and verification time. A repository on the same local node does not satisfy off-node disaster recovery.

15.1 Backup commands for the VERIFIED standalone stanza

```
kubectl -n "$NS" exec backup-lab-0 -- \
  gosu postgres pgbackrest --stanza=lab --type=full backup
kubectl -n "$NS" exec backup-lab-0 -- \
  gosu postgres pgbackrest --stanza=lab --type=diff backup
kubectl -n "$NS" exec backup-lab-0 -- \
  gosu postgres pgbackrest --stanza=lab --type=incr backup
```

16. Final Coverage Checklist

Logical backup and pg_basebackup sections remain in the original first eight pages. WAL checks and image validation are described in this revision. Full/incremental/differential backups are verified only for backup-lab-0. Restore and PITR commands use separate new PVCs and still require execution and SQL validation in your cluster. Live acid-postgres-ha GCS backup, retention, encryption and PITR remain gated on approved pod identity, repository setup, WAL archiving and an isolated restore drill. Do not describe the standalone lab backup as a live cluster backup.